

CODING STANDARDS & CONVENTION

Kurt, Tarik

Project NEIE NAOBER | ALFA COLLEGE

Table of Contents

PURPOSE.....	2
NAMING CONVENTION.....	2
VARIABLES	2
FUNCTIONS.....	2
CLASSES	2
CONSTANTS.....	2
FORMATTING	3
COMMENTS	3
FUNCTIONS.....	4
FILE AND FOLDER NAMING	4
CODE QUALITY	4
GIT CONVENTION	5
TECHNOLOGIES	5

PURPOSE

This document contains the rules of how we code.

The purpose of this document is to make the code clear, readable and consistent.

NAMING CONVENTION

Variables

Use `camelCase` for variable names.

```
let userName = "John";  
let totalPoints = 100;
```

Avoid:

```
let User_Name = "John";  
let total_points = 100;
```

Functions

Use `PascalCase` and use a descriptive name that explains what the function does.

```
function CalculateTotalPrice()  
{  
    // ...  
}
```

Classes

Use `PascalCase` for class names.

```
class UserAccount  
{  
    // ...  
}
```

Constants

Use `UPPER_SNAKE_CASE` for values that never change.

```
const MAX_LOGIN_ATTEMPTS = 3;
```

FORMATTING

Use 4 spaces for indentation.

Keep lines reasonably short.

Add spaces around operators.

Example:

```
if (user.isActive)
{
    console.log("User is active");
}
```

```
if (age <= 18 && hasLicense)
{
    //...;
}
```

Avoid:

```
if(age > 18){
// ...
}
```

COMMENTS

Comments should explain **why** something is done, not simply repeat what the code does.

Good:

```
// Prevents duplicate registrations from being created
if (existingUser) {
    return;
}
```

Avoid:

```
// Check if user exists
if (existingUser) {
    return;
}
```

FUNCTIONS

Functions should have one clear responsibility.

Good:

```
function CalculateTotalPrice(items)
{
    // Only calculates the total
}
```

Avoid functions that perform many unrelated tasks, such as calculating prices, sending emails, and updating the database in the same function.

FILE and FOLDER NAMING

Use descriptive names for files and folders.

Example:

```
src/
├── Components/
├── Services/
├── Models/
└── Utils/
```

Use PascalCase for both class/utility files:

```
UserProfile.js
LoginForm.js
```

```
ValidationUtils.js
DateUtils.js
```

CODE QUALITY

Team members should:

- Avoid duplicate code.
- Remove unused variables and functions.
- Use meaningful names.
- Keep functions small and focused.
- Avoid unnecessary complexity.
- Review their own code before committing it.

GIT CONVENTION

Commit messages should briefly describe the change.

Good:

```
Add user registration validation
Fix login error handling
Update database connection
```

Avoid:

```
stuff
changes
final
asdf
```

Each commit should contain one logic

TECHNOLOGIES

Frontend: React + JS

Backend: PHP (Laravel)

Database: MySQL

Authentication: JWT or session-based